



UNIVERSITY
OF COLOGNE

MOCK EXAM

Advanced Analytics and Applications

Agenda

1	Fill in the Blanks
2	Feed-Forward Neural Network
3	Support Vector Machines
4	Clustering
5	Reinforcement Learning
6	Python Task

Fill in the Blanks

In the exam there might be **blanks** or **single choice** questions, or a mix!

- a) semi-supervised** (Lecture 7, Slide 50)
- b) linearly** (Lecture 2, Slides 10ff)
- c) Gaussians / Gaussian Distributions** and **EM / Expectation Maximization** (Lecture 3)
- d) Discretization** (Lecture 4, Slide 56) and **Hierarchical** (Lecture 4, Slide 63 and Workshop Notebooks)
- e) Augmentation** (Lecture 7, Slides 40ff)
- f) Smaller** (Lecture 8, Slides 18ff)
- g) Actor-Critic** (Lecture 13, Slide 20)
- h) ϵ -greedy / epsilon-greedy** (Lecture 10, Slide 39)

Agenda

1

Fill in the Blanks

2

Feed-Forward Neural Network

3

Support Vector Machines

4

Clustering

5

Reinforcement Learning

6

Python Task

Question 2

Lecture 5

- (Initialize random weights and biases)
- Forward Pass:
 - Select (mini-)batch
 - run batch through current neural network (weights and biases) for prediction
 - Calculate loss
- Backward Pass:
 - Based on loss, compute gradient w.r.t. to each weight and bias
 - Use stochastic gradient descent to update weights and biases based on gradient with learning rate
- Repeat with batches until an epoch is finished, repeat for epochs until training converges (no change above specified threshold in validation error)

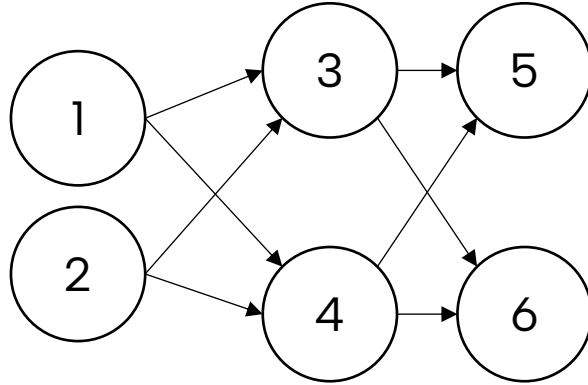
Question 3

$$w_{13} = -3, w_{35} = 2, w_{23} = 1, w_{45} = -3, w_{14} = 5, w_{36} = 3, w_{24} = -1, w_{46} = 2$$

$$g(x) = \begin{cases} 1 & \text{if } x \geq 0, \\ 0 & \text{otherwise.} \end{cases}$$

$$x = \sum_i w y_i$$

a)



Lecture 6, Slide 18

b) Input: $x_1 = 1, x_2 = 1$
First Layer:

$$\begin{aligned} y_3 &= g(w_{13}x_1 + w_{23}x_2), y_4 = g(w_{14}x_1 + w_{24}x_2) \\ y_3 &= g(-3 * 1 + 1 * 1), y_4 = g(5 * 1 - 1 * 1) \\ y_3 &= g(-2) = 0, y_4 = g(4) = 1 \end{aligned}$$

Second Layer (Output):

$$\begin{aligned} y_5 &= g(w_{35}x_3 + w_{45}x_4), y_6 = g(w_{36}x_3 + w_{46}x_4) \\ y_5 &= g(2 * 0 - 3 * 1), y_6 = g(3 * 0 + 2 * 1) \\ y_5 &= g(-3) = 0, y_6 = g(2) = 1 \end{aligned}$$

Agenda

1

Fill in the Blanks

2

Feed-Forward Neural Network

3

Support Vector Machines

4

Clustering

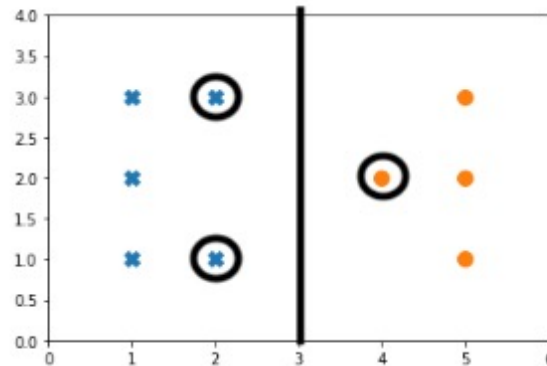
5

Reinforcement Learning

6

Python Task

- **Question 4:** The optimal linear separating boundary is exactly at $x = 3$. This boundary maximizes the margin / the distance between support vectors.



- **Question 5:**
 - *Option 1:* SVM Classification is not the right choice for multi-class problems, as support vector classifiers are fundamentally designed as binary classifiers.
 - *Option 2:* If we utilize non-linear kernels like RBF (because the data needs that) we lose the ability to directly interpret features – if there is a need to understand the impact of features, SVM might be bad.
 - *More answers exist (if probabilities are needed, if dimensionality is too large, etc)*

Agenda

1

Fill in the Blanks

2

Feed-Forward Neural Network

3

Support Vector Machines

4

Clustering

5

Reinforcement Learning

6

Python Task

- **Question 6:** We should opt for *soft-clustering* as there are apparent overlaps. Soft clustering would yield probabilities instead of hard assignments (as opposed to hard clustering), reflecting the associated uncertainty, which makes it better suited for the example.
- **Question 7:**
 - Gaussian Mixture Models (GMM) assign data points a probability of belonging to a cluster derived from a mixture of multiple Gaussians. That mixture needs parameters: The mixing weights, and the means and variances of the Gaussians. As there is no labeled data available (unsupervised) we can't find these parameters analytically. EM is an iterative approximation procedure for these parameters.
 - We initialize the GMM with random values for these parameters, often equal weights and evenly spaced means over the data with unit variance.
The EM algorithm then alternates between two steps:
It estimates each posterior probability of the current parameters' GMM having generated the data (E-Step) and then re-estimates the parameters such that they maximize the (log-)likelihood of the data points under current model (given the cluster probabilities from the E-step).

Agenda

- | | |
|----------|-------------------------------|
| 1 | Fill in the Blanks |
| 2 | Feed-Forward Neural Network |
| 3 | Support Vector Machines |
| 4 | Clustering |
| 5 | Reinforcement Learning |
| 6 | Python Task |

Question 8

a) You can write down either of the two (decomposed form or full form):

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

$$q_{\pi}(s, a) = r(s, a) + \gamma \sum_{s', a'} p(s' | s, a) \pi(a' | s') q_{\pi}(s', a')$$

Lecture 11, Slide 18

b) SARSA:

We look for the first occurrence of $A_t = c$

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha [R_{t+1} + \gamma q(S_{t+1}, A_{t+1}) - q(S_t, A_t)]$$

$$q(x, c) \leftarrow q(x, c) + 0.1 [6 + 0.5 * q(x, b) - q(x, c)]$$

$$q(x, c) \leftarrow 2 + 0.1 [6 + 0.5 * 1 - 2] = 2.45$$

As π is probabilistic, we assume that $A_{t+1} = b$

Lecture 12, Slide 9

Q-Learning:

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a (q(S_{t+1}, a) - q(S_t, A_t))]$$

$$q(x, c) \leftarrow q(x, c) + 0.1 [6 + 0.5 * q(x, c) - q(x, c)]$$

$$q(x, c) \leftarrow 2 + 0.1 [6 + 0.5 * 2 - 2] = 2.5$$

This time we go for the maximizing action which is (given our current knowledge), c

Lecture 12, Slide 11

Question 9

- Initialize replay buffer empty, set update threshold U , training threshold n , stopping criterion L
- Initialize both networks with the same random weights/biases $\theta = \hat{\theta}$
- For each episode: Initialize starting state s
 - For each step t
 - Select random action with probability ϵ , otherwise select action that $\max_a Q(s, a, \theta)$
 - Execute a and observe s', r , store (s, a, r, s') in replay buffer
 - Every n steps:
Sample mini batch from replay buffer and calculate TD target ($y_j = r_j + \gamma * \max_{a'} Q(s'_j, a', \hat{\theta})$)
 - Perform stochastic gradient descent step on error metric of TD error (e.g. MSE where k is batch size) ($\frac{1}{k} \sum_k (y_j - Q(s_j, a_j, \theta))^2$)
 - If $t \% U = 0$ (roughly, account for episodes), update $\hat{\theta} \leftarrow \theta$
 - Update $s \leftarrow s'$
 - If error metric $< L$, stop (convergence of values, not necessarily just networks!)

Lecture 12, Slide 41

Question 10

- **Value-based** methods learn a value function and act based on an implicit policy such as ϵ -greedy
- **Policy-based** methods learn no value function but directly a (probabilistic) policy and act on this explicit policy
- **Actor-critic** methods combine these two, as they learn a policy (the actor) and value functions (the critic). The actor acts on the explicit policy it learns, but updates it according to the critics learned value.

Agenda

- 1 Fill in the Blanks
- 2 Feed-Forward Neural Network
- 3 Support Vector Machines
- 4 Clustering
- 5 Reinforcement Learning
- 6 Python Task**

Python / Spatial Analysis

a) This is a **choropleth** map (colored polygons/shapes based on values and colorscale). In this case the count of cities within one hexagon is the value that is used to color shapes based on a colorscale that has increasing hue/darkness with increasing value. This type of maps presents visual comparison between areas/shapes on a map.

b) Line 1: `import geopandas as gpd`
Line 7: `cities = gpd.read_file(cities_path, mask=world[world.continent == "Europe"])`
Line 10: `agg.plot(ax=ax, alpha=0.7, column='cities', cmap='Reds')`

Something below 1

Something that makes sense

c) One would need to do a **spatial join / sjoin** with the intersect/contains relationship test **between the cities and the world dataset** to obtain the respective country for each city. After that a choropleth map can be created using the same logic.

Contact



For general questions and enquiries on **research, teaching, job openings** and **new projects** refer to our website at www.is3.uni-koeln.de



For specific **enquiries regarding this course** contact us by sending an email to the **IS3 teaching address** at is3-teaching@wiso.uni-koeln.de

To help us process your request efficiently, please use the following subject line format:

[AAA] <Your request subject>